



18BIG

Big Data Tools & Architecture

Lecture 02 — Data Sources & Bronze Ingestion

www.bighub.ai

Your guides to applied AI



■ Recap: last time

Three questions from **Session 01 — Foundations**:

1. What does a **Lakehouse** give you that a plain Data Lake doesn't?
2. **OLTP vs. OLAP** — which one is our e-shop database, and which one is the lakehouse?
3. What goes wrong with a money transfer **without ACID**?

Warm-up — answer out loud before we build on it.

■ Today

- **Medallion** — Bronze → Silver → Gold, and why layers
- **Where data comes from** — source types
- **How** — batch vs. streaming, pull vs. push, full vs. incremental
- **Idempotence** — the one property that matters most
- **The Bronze layer** — land raw data, add lineage
- **Hands-on** — build `eshop ingest` into Bronze
- Preview of **CDC** (full detail in Session 10)



Part 1

The big picture: medallion

Your guides to applied AI



■ The medallion architecture



- Data flows **one way**, and each layer **raises quality** a step.
- Every layer is **rebuildable** from the one before it, all the way back to Bronze.
- Today we build the **first box**: Bronze.

Medallion is a **convention**, not a product: raw / staged / curated are the same idea under other names.

■ Three layers at a glance

Layer	Data state	Typical users	Purpose	Our project
Bronze	raw, as landed, duplicates & errors included, + metadata	data engineers, support	ingestion, archive, reprocessing	lake/bronze/ · today
Silver	cleaned, typed, deduplicated, per object, not yet joined across sources	engineers, analysts, data scientists	self-service analytics, ML, object-level truth	lake/silver/ · S4–S5
Gold	aggregated, denormalized, read-optimized (star schema, marts)	business analysts, decision makers, BI tools	dashboards, KPIs, data products	lake/gold/ · S6–S7

■ Why bother with layers?

What you get

- **Reprocess** from raw without hitting the sources again
- **Separation of concerns**: ingest $\neq$ clean $\neq$ model
- Tests & owners **per layer**
- **Debugging**: find the layer where a bug crept in
- Access control: not everyone sees raw data

What it costs

- Data stored **several times** (storage is cheap)
- More pipelines to run and monitor
- Some teams add layers (**landing**, **platinum**), small ones merge them

Deeper dive on layers & storage in Session 03



Part 2

Where does data come from?

Your guides to applied AI



■ Three families of sources

Files / object storage

CSV, JSON, Parquet on
S3/ADLS/local

Databases (JDBC)

operational systems, snapshots
& queries

Queues / streams

Kafka, Event Hub, Redpanda

- Most platforms ingest from **several** of these at once — each with its own cadence.
- Today: **files + a database**. Streams come back in **Session 10**.

■ APIs, SaaS & the messy rest

REST APIs

- In-house apps, legacy systems, third parties
- **Pagination** — cursor beats offset
- **Rate limits** → back off & retry
- **Auth** — API keys, OAuth tokens

SaaS & spreadsheets

- Salesforce & co. → **connectors** (Fivetran, Airbyte)
- **Excel** on SharePoint / OneDrive — finance still runs on it
- Dates, formulas, meaning hidden in *formatting*

Every source type has its own failure modes — Bronze keeps what arrived so you can fix it later.

■ In our project

Files (CSV)

- `customers.csv`
- `products.csv`
- `payments.csv`

DuckDB reads both directly — no server, no driver setup

Database (JDBC-like)

- `shop.sqlite` → `orders`, `order_items`
- SQLite plays the **operational system**



Part 3

Integration patterns

Your guides to applied AI



■ Batch vs. streaming

Batch

- Pull files / DB snapshots on a **schedule**
- Cheapest way to move large volumes
- Latency: minutes → hours

Streaming

- Events land within **seconds**
- More moving parts (broker, monitoring)
- Costs **~3–5×** a batch setup

Rule of thumb: **batch by default**, stream only where the business truly needs low latency.

■ Pull vs. push

Pull

- The **platform** fetches on a schedule
- Queries, file listings, API polling
- + you control timing & volume
- — latency = schedule interval

Push

- The **source** sends data when it happens
- **Webhooks** (HTTP POST), storage events, streams
- + near real-time
- — you need an endpoint, auth, **retries & dedup**

Today: pull — we read the files and the database ourselves

■ Delivery guarantees

Push & streams: networks fail, senders retry. What does the receiver get?

Guarantee	Meaning	Risk	Typical where
At-most-once	fire & forget, no retry	lost messages	metrics, logs you can afford to drop
At-least-once	retry until acknowledged	duplicates	webhooks, Kafka default, most queues
Exactly-once	each message has effect once	cost & complexity	Kafka transactions, Flink + transactional sinks

In practice: **at-least-once + an idempotent receiver** = *effectively* exactly-once. That's why idempotence matters.

■ Full vs. incremental load

	Full load	Incremental load
What moves	the whole dataset every run	only new/changed rows
Cost	high, grows with data	low, stays flat
Complexity	trivial	needs a change key / watermark
Good for	small/reference tables	large, frequently-updated tables

We'll do a **full** Bronze load today, then an **incremental** one as the harder task.

■ Incremental from SQL: watermarks

```
-- full load
SELECT * FROM orders

-- incremental: remember the last value you loaded (one watermark per table)
SELECT * FROM orders
WHERE created_at > :last_watermark OR updated_at > :last_watermark;

-- append-only tables: a growing id works too
SELECT * FROM orders WHERE order_id > :last_id;
```

- Needs a **reliable change column**, and it cannot see **deletes** (next slide).
- **No change column → no incremental query → you're back to a full load.**

■ Deletes: soft vs. hard

Soft delete

- Row stays, gets flagged: `is_deleted = true`, `deleted_at = now()`
- It's just an **update** → a watermark on `updated_at` **catches it**
- Silver filters it out (or keeps it for history)

Hard delete

- `DELETE FROM orders ...` → the row is **gone**
- A watermark query **never sees it**
- **Effectively rules out incremental load → full load** (compare keys), unless you have **CDC**

Ask the source team early: **"How do you delete?"** Hard deletes are a common cause of lakehouse numbers that don't match the source.

■ Idempotence — the property that matters most

Design every ingestion job by asking "**what if this runs twice?**" *first*, and the happy path second.

- Re-running must **not** duplicate or corrupt data.
- Enables safe **retries, backfills and replays** (we'll lean on this with the orchestrator).
- Our way: a **watermark** — a second run finds nothing above it and lands **zero** rows.
- Bronze doesn't compare rows; **deduplication happens in Silver**.

■ Change Data Capture (preview)

Instead of re-reading a whole table, **CDC** streams just the inserts/updates/deletes by reading the database's **transaction log**:



The gold standard for incremental ingestion from operational DBs. **Full detail in Session 10.**



Part 4

The Bronze layer

Your guides to applied AI



■ What Bronze is

"What arrived, we don't touch." Landing/Bronze is the raw snapshot — the **source of truth** you can always reprocess from.

- Store raw in the **original source format**: CSV stays CSV, JSON stays JSON. Any processing can **lose information** (leading zeros, time zones, encodings, unknown JSON fields) → **no reproducibility**.
- **Never delete**, archive instead; keep duplicates and bad rows: Bronze is the **audit trail**.
- **Schema-on-read**: no schema enforced on the way in; Silver applies types & rules.
- Don't clean or reshape here; add only **technical metadata**.

Our project: originals stay in data/raw/ (landing) · Bronze = Parquet copy + metadata

■ Lineage metadata we attach

```
df.with_columns(  
    pl.concat_str(business_cols, separator="|").hash(0).cast(pl.Utf8).alias("_row_hash"),  
    pl.lit(datetime.now()).alias("_ingested_at"),  
    pl.lit(source).alias("_source"),  
    pl.lit(batch_id).alias("_batch_id"),  
)
```

Standard bronze columns: ingestion timestamp · source identifier · batch id · a content hash as an audit column (Silver can use it to deduplicate).

■ Bronze best practices (2026)

- **Never delete** raw data — archive to cold storage (in reality **price** is a concern)
- **Partition by ingestion date** (Session 3)
- no schema enforcement
- Prefer a **parquet format**
- Keep **metadata + lineage** on every row

■ Metadata-driven ingestion

```
# src/eshop/ingestion/sources.yml
sources:
  - name: customers
    type: csv
    file: customers.csv
    load: full
  - name: orders
    type: sqlite
    table: orders
    load: incremental
    watermark: order_id
```

- **One generic loader** + a **source catalogue**
- New source, or full → incremental = a **config change**, not new code
- Watermark state: here derived from Bronze; bigger frameworks keep a **control table** (last watermark, last run, status)
- **100 tables ≠ 100 pipelines**



Hands-on

Land the sources in Bronze

Your guides to applied AI



■ Run it

```
git pull && uv sync
just datagen          # source data (if you don't have it yet)
just ingest           # land every source into data/lake/bronze/
just bronze orders    # row count, batches, last rows: note the _ columns
```

Test the incremental load:

```
just ingest           # 2nd run: orders → 0 rows
just new-orders 5     # the source gets 5 new orders
just ingest           # orders → exactly 5 rows, in a new batch
```

■ Today's exercise

Fill in the `# TODO` s in `src/eshop/ingestion/bronze.py` :

1. Add the lineage columns `_ingested_at` / `_source` / `_batch_id` .
2. Read a SQLite table via DuckDB's `sqlite_scan` (full load).
3. **Incremental ingest**: compute the **watermark** (max `order_id` in Bronze), read only newer rows.
4. **Incremental ingest: append** them to Bronze; a re-run must land **zero** rows.
5. **(metadata-driven)** Switch `order_items` to incremental in `sources.yml` , no code change.

■ Key takeaways

- **Medallion:** Bronze (raw) → Silver (clean) → Gold (business); **Bronze = raw source of truth** + lineage metadata.
- Data comes from **files, databases, APIs and streams**, usually several at once.
- **Batch/pull by default**; stream or push only when latency demands it.
- Incremental needs a **watermark**; no change column or hard deletes → **full load**. **CDC** streams only changes (Session 10).
- **Idempotence first**: always ask *"what if this runs twice?"*.

■ Next time

Session 03 — Parquet & the medallion architecture

- Why columnar storage powers analytics (row groups, compression, pushdown)
- **Partitioning** and partition pruning — measure it yourself
- File format vs. table format — the road to Delta

■ References & further reading

- Medallion architecture, 2026 guide — dataforest.ai/blog/medallion-architecture
- Data ingestion patterns: batch, streaming & CDC — datavidhya.com
- API pagination patterns — bytebytego.com
- Debezium (change data capture) — debezium.io
- Delta Lake (ACID table format) — delta.io

The background of the slide features a series of concentric, wavy lines in various shades of purple, creating a sense of depth and movement.

bighub

Thank you

Let's build.

www.bighub.ai

Your guides to applied AI

